WYBRANE ALGORYTMY ROZWIĄZYWANIA PROBLEMU IMPASÓW W SYSTEMACH PROCESÓW WSPÓŁBIEŻNYCH

Robert Wójcik

Politechnika Wrocławska

1. Impasy w systemach procesów współbieżnych
2. Klasyczne algorytmy rozwiązywania problemu impasów
3. Podsumowanie

Systemy współbieżnych procesów sekwencyjnych z wzajemnym wykluczaniem

Założenia:

- wzajemne wykluczanie w dostępie do wspólnych zasobów,
- jednostkowe żądania zasobowe,
- przetrzymywanie zasobu aż do momentu rezerwacji kolejnego zasobu,
- występowanie ograniczeń zasobowych.

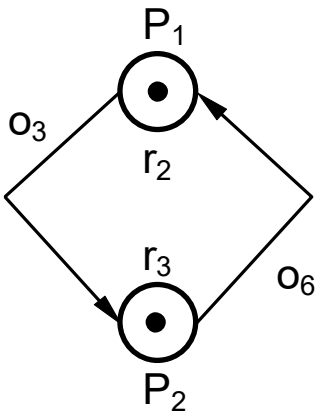
Przykłady.

$P_1 = (o_1,r_1)\ (o_2,r_2)\ (o_3,r_3)$ - proces sekwencyjny nr 1.

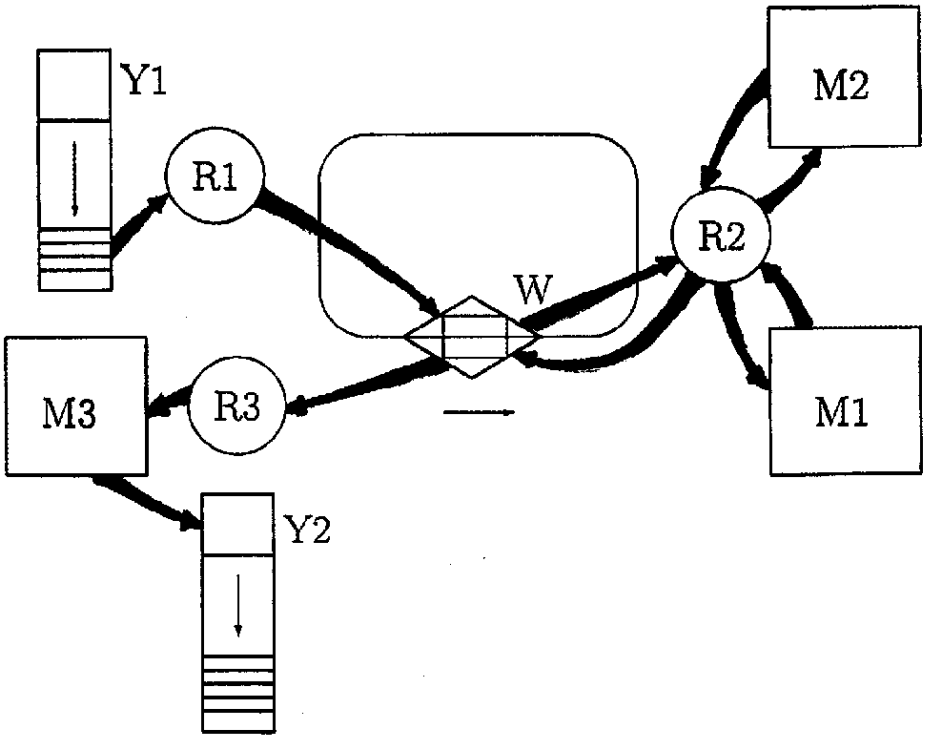
$P_2 = (o_4,r_4)\ (o_5,r_3)\ (o_6,r_2)$ - proces sekwencyjny nr 2.

Przy założeniu jednostkowych pojemności zasobów w systemie może pojawić się stan blokady:

P_1 - zakończył operację (o_2,r_2),
P_2 - zakończył operację (o_5,r_3).



System EGP (Elastyczne Gniazdo Produkcyjne)



Niech

$$ZP = \{ P_i \mid i = 1,...,n \}$$

zbiór zadań (procesów) realizowanych w systemie EGP.

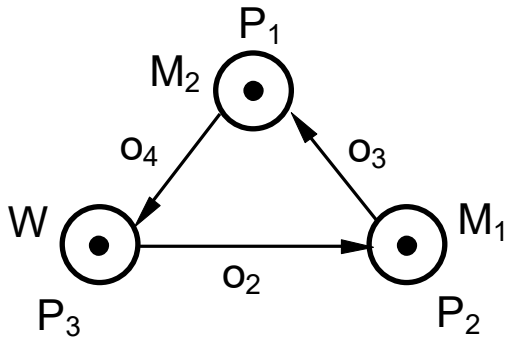
Każdy proces ma postać:

$$P_i = (o_1,W)\ (o_2,M_1)\ (o_3,M_2)\ (o_4,W)\ (o_5,M_3)\ .$$

Stan blokady w systemie:

P_1 - zakończył (o_3,M_2),
P_2 - zakończył (o_2,M_1),
P_3 - zakończył (o_1,W).

Pojemności zasobów: W, M_1, M_2, M_3 równe 1.



Stan systemu procesów

Niech

$R = \{ r_i \mid i = 1,...,m \}$ - zbiór zasobów,
$c(r_i)$ - liczba jednostek zasobu r_i .

$OP = \{ o_i \mid i = 1,...,L \}$ - zbiór operacji realizowanych przez procesy.

Stan systemu: $S = [s(1),...,s(i),...,s(L)]$,

gdzie s(i) - liczba procesów realizujących operację $o_i \in OP$.

Dla systemu EGP: $S = [s(1), s(2), s(3), s(4), s(5)]$.

Stan początkowy: $S_0 = [0,0,0,0,0]$.
Stan blokady: $S_b = [1,1,1,0,0]$.

Przestrzeń stanów - zbiór stanów osiągalnych ze stanu początkowego $S_0 = [0,...,0]$.

Stan blokady (impasu) – stan, w którym istnieje zbiór procesów, dla których nie można już wykonać żadnego przydziału zasobowego.

Stany niebezpieczne - stany, z których po skończonej sekwencji przydziałów osiągany jest stan blokady.

Stany zabronione - stany niebezpieczne lub stany blokady.

Można pokazać, że w przypadku systemu EGP możliwe są 22 stany z czego 2 są stanami niedozwolonymi (jeden stan niebezpieczny i jeden stan blokady całkowitej).

Przestrzeń stanów systemu EGP

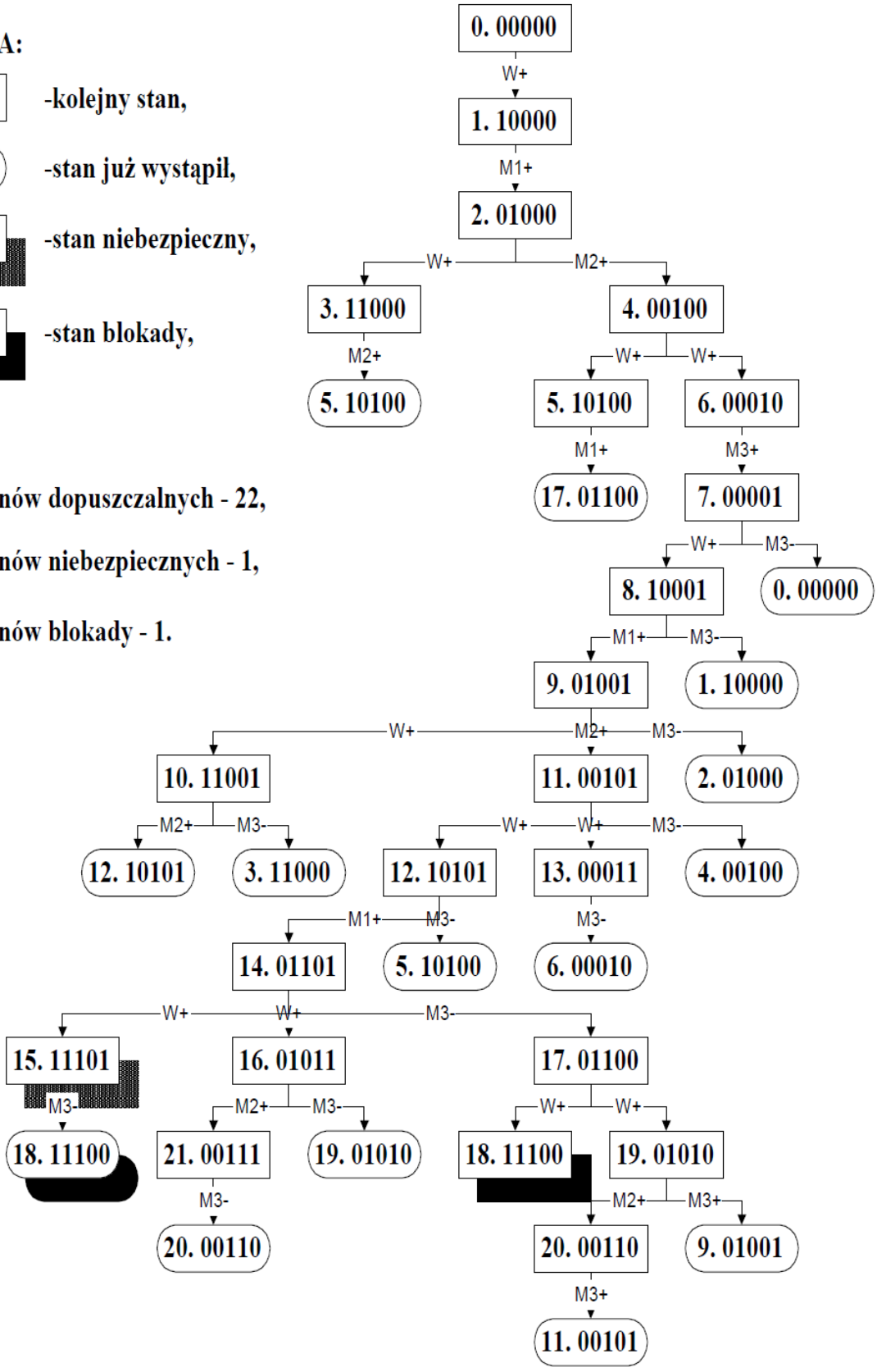
LEGENDA:

xx. xxxxx	-kolejny stan,
xx. xxxxx	-stan już wystąpił,
15. 11101	-stan niebezpieczny,
18. 11100	-stan blokady,

Liczba stanów dopuszczalnych - 22,

Liczba stanów niebezpiecznych - 1,

Liczba stanów blokady - 1.



Złożoność obliczeniowa problemu

Problem unikania impasów sprowadza się do wykrywania i eliminacji stanów zabronionych.

W ogólnym przypadku jest to problem trudny obliczeniowo (NP-zupełny).

Stan badań

- Rozwiązania (algorytmy) opracowane dla ogólnego przypadku oparte są na warunkach *wystarczających* dla unikania blokad, tzn. mogą odrzucać niektóre stany poprawne.

- Przy uwzględnieniu pewnych dodatkowych założeń można podać warunki *wystarczające i konieczne* dla unikania blokad (stanowią podstawę algorytmów maksymalnie dopuszczających).

Klasyczne algorytmy rozwiązywania problemu impasów

1. Algorytm oparty na koncepcji „wszystko albo nic”.
2. Algorytm bankiera.
3. Algorytm oparty na koncepcji stref synchronizacji.
4. Algorytm wykorzystujący graf żądań zasobowych.

Kryteria oceny algorytmów

- Liczba elementów przestrzeni stanów akceptowalnych.
- Złożoność obliczeniowa.

Algorytm „wszystko albo nic”

A_i - zbiór zasobów wykorzystywanych przez proces P_i .

Reguła alokacji zasobów

Proces P_i może rozpocząć wykonywanie operacji jeżeli zarezerwuje wszystkie zasoby należące do zbioru A_i .

Złożoność obliczeniowa testu: O(m) ,

gdzie m = |R| - liczba zasobów.

Przykładowe protokoły:

1. Algorytm Dekkera - synchronizacja dwóch procesów.
2. Algorytm Pettersona – synchronizacja dwóch procesów
3. Algorytm Dijkstry - synchronizacja n procesów.

Literatura:

Raynal M., *Algorithms for Mutual Exclusion*, The MIT Press Cambridge, Massachusetts, 1986.

Stany akceptowalne - algorytm „wszystko albo nic”

LEGENDA:

xx. xxxxx -kolejny stan,

xx. xxxxx -stan akceptowalny,

xx. xxxxx -stan już wystąpił,

15. 11101 -stan niebezpieczny,

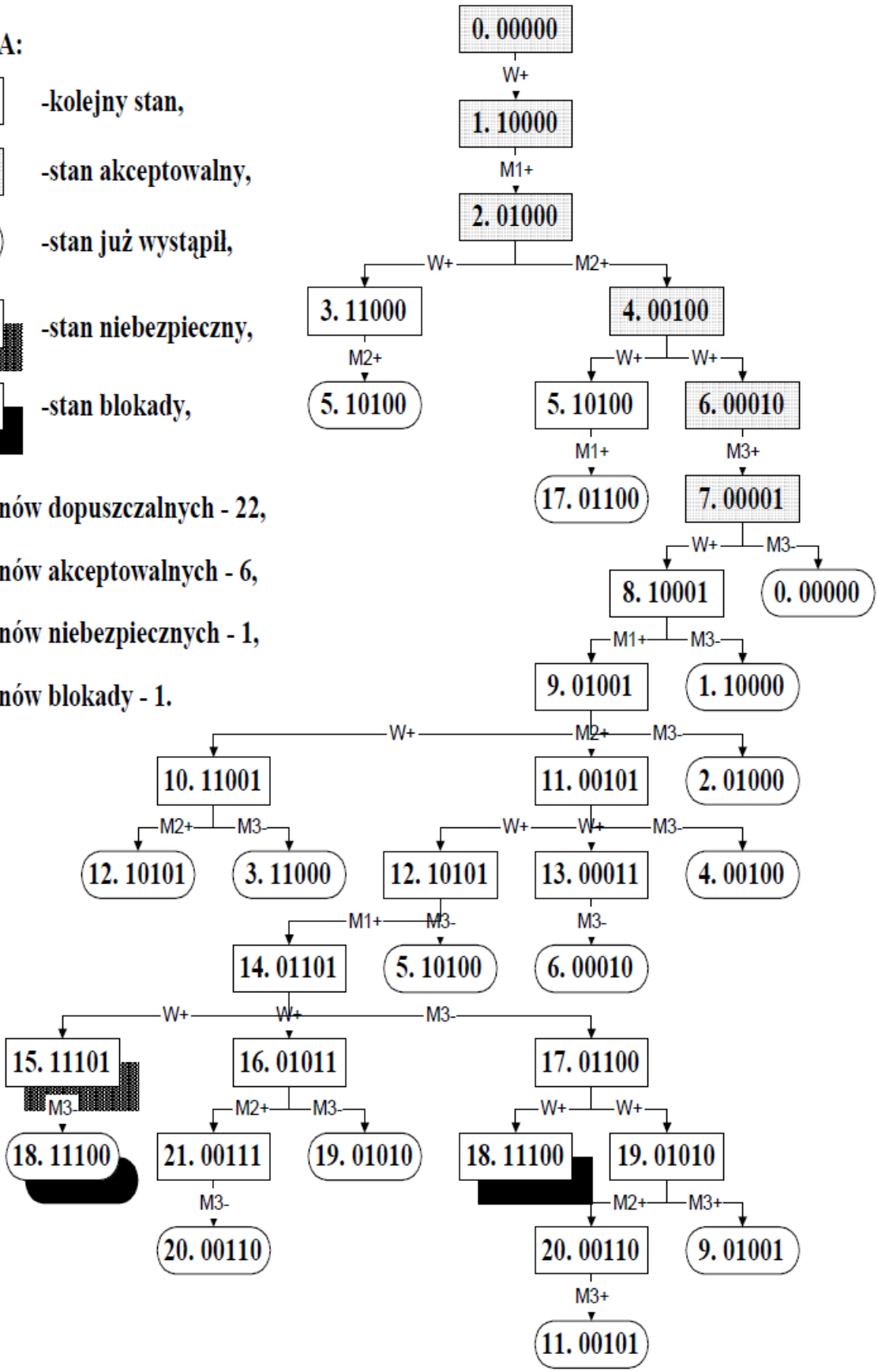
18. 11100 -stan blokady,

Liczba stanów dopuszczalnych - 22,

Liczba stanów akceptowalnych - 6,

Liczba stanów niebezpiecznych - 1,

Liczba stanów blokady - 1.



Algorytm bankiera

$c(r_i)$ - liczba jednostek zasobu r_i ,
$mA(r_i,P_j)$ - maksymalna liczba jednostek zasobu r_i wykorzystywana przez proces P_j .

Reguła alokacji zasobów

Proces P_j może zarezerwować żądane zasoby, jeżeli w wyniku rezerwacji pojawi się stan, w którym zbiór aktywnych procesów można uporządkować w taki sposób, aby możliwe było spełnienie maksymalnych żądań zasobowych procesów.

Przykład. Jeden zasób w systemie: $c(r) = 10$. Maksymalne żądania procesów: $mA(r,P_1) = 8$, $mA(r,P_2) = 6$

Stan odrzucany	Stan akceptowany
P_1 - żąda 5 jednostek	P_1 - żąda 4 jednostki
P_2 - żąda 3 jednostki	P_2 - żąda 5 jednostek
Bankier - wolne 2 jednostki	Bankier - wolna 1 jednostka

Złożoność obliczeniowa testu: $O(mn^2)$,

gdzie m - liczba zasobów, n - liczba procesów.

Literatura:

Peterson L., Silberschatz A., *Operating Systems Concepts*, Addison-Wesley, Amsterdam, 1983.

W przypadku systemu EGP pojemności zasobów
W, M_1, M_2, M_3 są równe 1.

Maksymalne żądania zasobowe procesów, dotyczące każdego zasobu są więc równe 1.

W klasycznym algorytmie Bankiera zakłada się, że w dowolnym stanie proces p_i może zgłosić maksymalne żądania zasobowe dotyczące każdego zasobu, który wykorzystuje w programie.

Podczas podejmowania decyzji o przydziale zasobowym nie jest brana pod uwagę informacja o strukturze sekwencyjnych programów, a jedynie informacja o maksymalnych żądaniach zasobowych występujących w całym programie.

Stąd w danym stanie systemu uwzględnia się możliwość zażądania przez proces wszystkich zasobów wykorzystywanych w całym programie w maksymalnych ilościach, w tym nawet tych zasobów, które były wykorzystywane na już zrealizowanych etapach programu sekwencyjnego.

W takim przypadku stan

S_i = [s(1), s(2), s(3), s(4), s(5)] = [1, 1, 0, 0, 0]

(W, M_1, M_2, W, M_3)

zostanie uznany przez klasyczny algorytm Bankiera za niebezpieczny, gdyż proces realizujący operację s(2) korzysta z zasobu M_1 i teoretycznie może zażądać każdego z potrzebnych mu w programie zasobów, tj. W, M_1, M_2, M_3. Podobnie proces realizujący operację s(1) korzysta z zasobu W i teoretycznie może zażądać każdego z potrzebnych mu w programie zasobów, tj. W, M_1, M_2, M_3. W przypadku, gdy pojemności zasobów są równe jeden żadne z tych żądań nie może być spełnione.

Podobna sytuacja zachodzi w przypadku stanu

S_j = [s(1), s(2), s(3), s(4), s(5)] = [1, 0, 0, 0, 1].

Dla procesu, realizującego operację s(5) nie jest uwzględniana informacja o tym, że proces ten kończy program i dlatego w stanie S_j Bankier przyjmuje, że w kolejnym kroku proces może żądać przydziału zasobów W, M_1, M_2, M_3, podobnie jak proces realizujący operację s(1). Stąd stan ten zostanie uznany przez klasyczny algorytm Bankiera za niebezpieczny.

W ogólnym przypadku, w systemach, w których pojemności wszystkich zasobów są równe 1 klasyczny algorytm Bankiera będzie zawsze dopuszczał w jednym programie sekwencyjnym do realizacji tylko jeden proces, tj. będzie dopuszczał alokacje zasobów zgodnie z regułą „wszystko albo nic”.

W przypadku programów sekwencyjnych znana jest struktura programów i może ona być uwzględniona podczas wykonywania alokacji zasobowych.

Proponowana modyfikacja polega na uwzględnianiu w każdym stanie systemu maksymalnych żądań zasobowych procesów tylko w ramach niezrealizowanych fragmentów ich programów sekwencyjnych, a nie tak jak w klasycznym algorytmie Bankiera zawsze żądań zasobowych dotyczących całego programu realizowanego przez proces.

W tym podejściu nadal stan

S_i = [s(1), s(2), s(3), s(4), s(5)] = [1, 1, 0, 0, 0]

zostanie uznany za niebezpieczny i odrzucony. Jednak już stan

S_j = [s(1), s(2), s(3), s(4), s(5)] = [1, 0, 0, 0, 1]

zostanie uznany za bezpieczny (otrzymaną przestrzeń stanów dopuszczalnych pokazano na kolejnym rysunku).

Stany akceptowalne – zmodyfikowany algorytm bankiera

LEGENDA:

xx. xxxxx -kolejny stan,

xx. xxxxx -stan akceptowalny,

xx. xxxxx -stan już wystąpił,

15. 11101 -stan niebezpieczny,

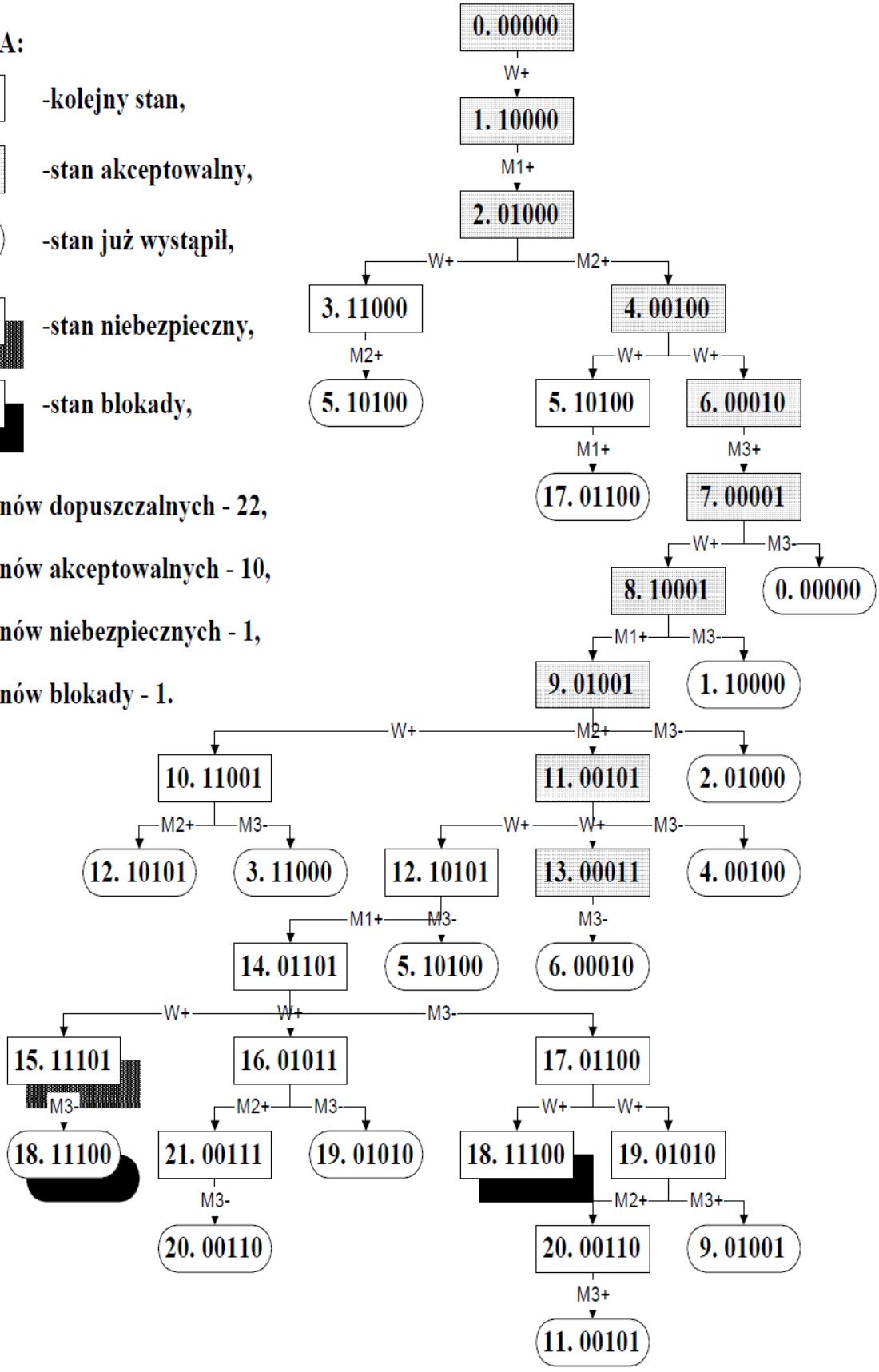
18. 11100 -stan blokady,

Liczba stanów dopuszczalnych - 22,

Liczba stanów akceptowalnych - 10,

Liczba stanów niebezpiecznych - 1,

Liczba stanów blokady - 1.



Algorytm strefowy

RS - zbiór zasobów, z których każdy wykorzystywany jest przez co najmniej dwie operacje (zasoby powtarzalne),

RU - zbiór zasobów, z których każdy wykorzystywany jest przez tylko jedną operację (zasoby niepowtarzalne),

OS - zbiór operacji korzystających z zasobów RS,

OU - zbiór operacji korzystających z zasobów RU.

Strefa synchronizacji - ciąg operacji tego samego rodzaju, tzn. operacji klasy OS lub OU.

Zbiór stref: $Z = ZS \cup ZU$ & $ZS \cap ZU = \varnothing$, gdzie

ZS - zbiór stref złożonych z operacji OS (s. powtarzalne),

ZU - zbiór stref złożonych z operacji OU (s. niepowtarzalne).

Dla systemu EGP:

$$P_i = (o_1,W)\ (o_2,M_1)\ (o_3,M_2)\ (o_4,W)\ (o_5,M_3)\ ,$$

$RS = \{ W \}$, $RU = \{ M_1, M_2, M_3 \}$,
$OS = \{ o_1, o_4 \}$, $OU = \{ o_2, o_3, o_5 \}$,
$ZS = \{ z_1, z_3 \}$, $ZU = \{ z_2, z_4 \}$,

$| o_1 | o_2\ o_3 | o_4 | o_5 |$ - podział na strefy .

z_1	z_2	z_3	z_4
S	U	S	U

Algorytm strefowy

Reguły alokacji zasobów

Proces P_i może wykonać operację o_j w strefie *powtarzalnej* $z_a \in ZS$, jeżeli równocześnie spełnione są warunki:

- żądany zasób jest dostępny ,
- każdy zasób, wykorzystywany przez operacje następujące po o_j w strefie z_a, posiada co najmniej jedną wolną jednostkę,
- w strefie niepowtarzalnej $z_b \in ZU$, następującej po strefie powtarzalnej $z_a \in ZS$, istnieje operacja korzystająca z zasobu posiadającego co najmniej jedną wolną jednostkę.

Proces P_i może wykonać operację o_j w strefie *niepowtarzalnej* $z_b \in ZU$, jeżeli żądany zasób jest dostępny.

Złożoność obliczeniowa testu: $O(L)$,

gdzie L - liczba operacji realizowanych przez procesy.

Literatura:

Banaszak Z., Krogh B., *Deadlock Avoidance in Flexible Manufacturing Systems* ... , IEEE Trans. On Rob. and Automation, 1990, Vol.6, No.6.

Stany akceptowalne - algorytm strefowy

LEGENDA:

xx. xxxxx -kolejny stan,

xx. xxxxx -stan akceptowalny,

xx. xxxxx -stan już wystąpił,

15. 11101 -stan niebezpieczny,

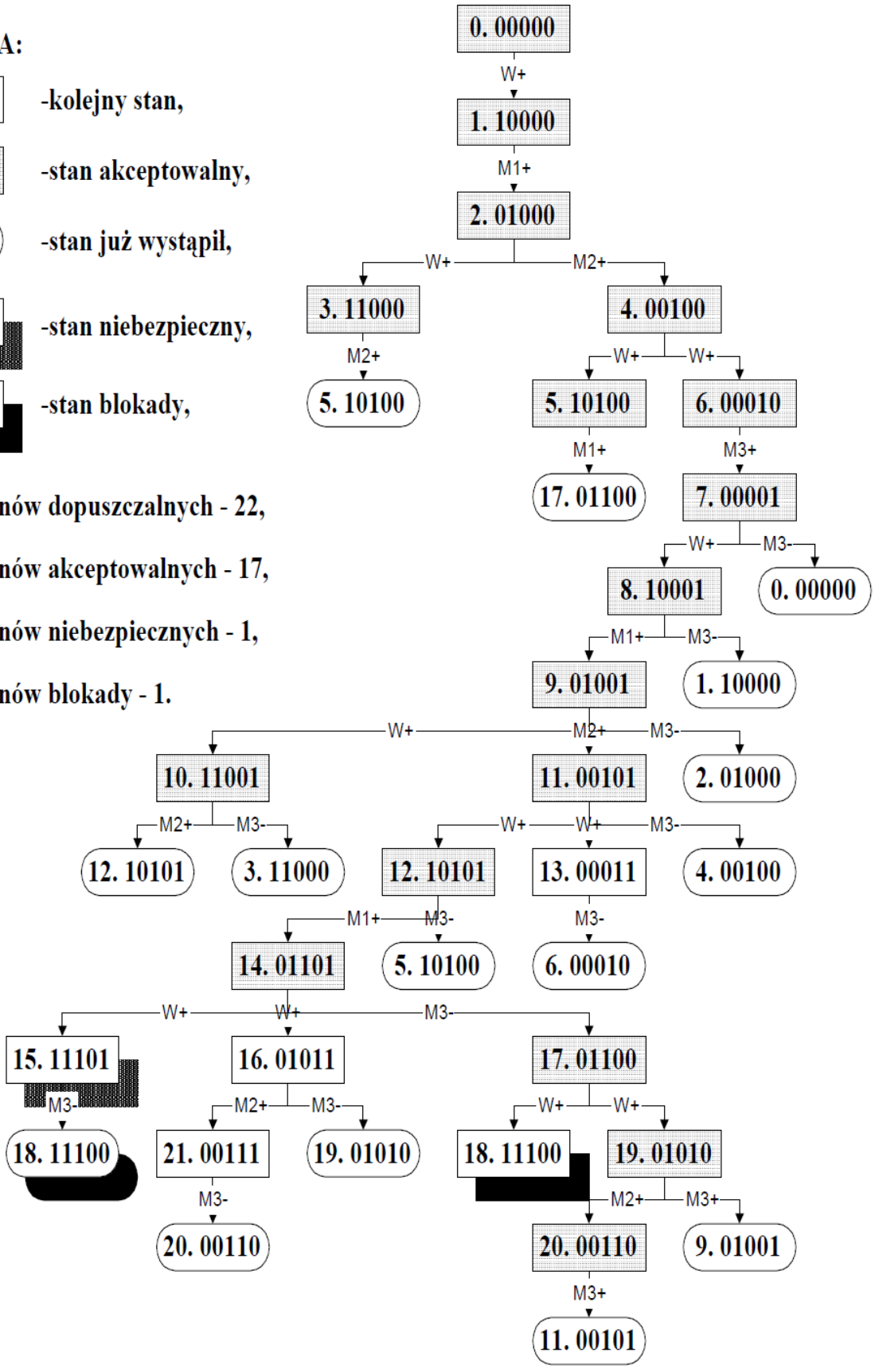
18. 11100 -stan blokady,

Liczba stanów dopuszczalnych - 22,

Liczba stanów akceptowalnych - 17,

Liczba stanów niebezpiecznych - 1,

Liczba stanów blokady - 1.



Algorytm grafowy

G = (R, E) - graf żądań zasobowych, gdzie

$R = \{ r_i \mid i = 1,...,m \}$ - zbiór zasobów,

L - liczba operacji,

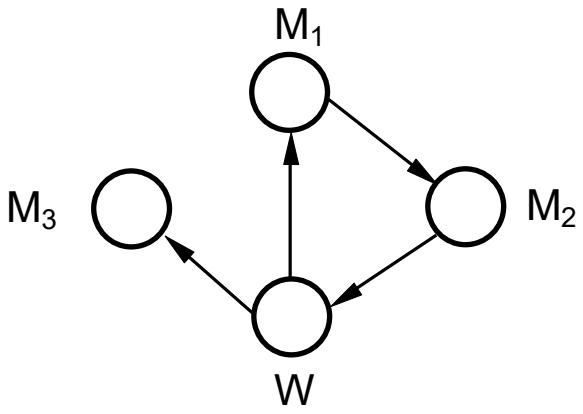
$E = \{ (x,y) \mid \exists\, k \in \{1,...,L-1\}\ (o_k, r_i)\ \&\ (o_{k+1}, r_j)\ \&\ \&\ (x = r_i)\ \&\ (y = r_j) \}$ - zbiór krawędzi.

W przypadku systemu EGP:

$P_i = (o_1,W)\ (o_2,M_1)\ (o_3,M_2)\ (o_4,W)\ (o_5,M_3)$,

$R = \{ W, M_1, M_2, M_3 \}$.

Graf żądań zasobowych



H - zbiór zasobów należących do cyklu o najmniejszej „pojemności”,

$c(r_i)$ - liczba jednostek zasobu r_i,

$c(H) = \sum c(r_i)\ \&\ r_i \in H$ - pojemność „minimalnego” cyklu.

Dla systemu EGP: $H = \{ W, M_1, M_2 \}$ & $c(H) = 3$.

Algorytm grafowy

c(H) - pojemność „minimalnego” cyklu,

d(S) - liczba procesów (zadań) realizowanych w systemie w stanie S.

Reguła alokacji zasobów

W stanie S_k przydział zasobu do procesu jest dozwolony, jeżeli w osiąganym stanie S_{k+1} spełniony jest warunek

$$d(S_{k+1}) \leq c(H) - 1 \quad .$$

W przypadku systemu EGP: $d(S_{k+1}) \leq 2$.

Złożoność obliczeniowa testu: $O(1)$.

Literatura:

Peterson L., Silberschatz A., *Operating Systems Concepts*, Addison-Wesley, Amsterdam, 1983.

Fanti M.P., Maione B., Mascolo S., Turchiano B., *Event Based Feedback Control for Deadlock Avoidance in Flexible Production Systems*, IEEE Trans. On Rob. and Automation, 1997, Vol.13, No.3.

Stany akceptowalne - algorytm grafowy

LEGENDA:

xx. xxxxx -kolejny stan,

xx. xxxxx -stan akceptowalny,

xx. xxxxx -stan już wystąpił,

15. 11101 -stan niebezpieczny,

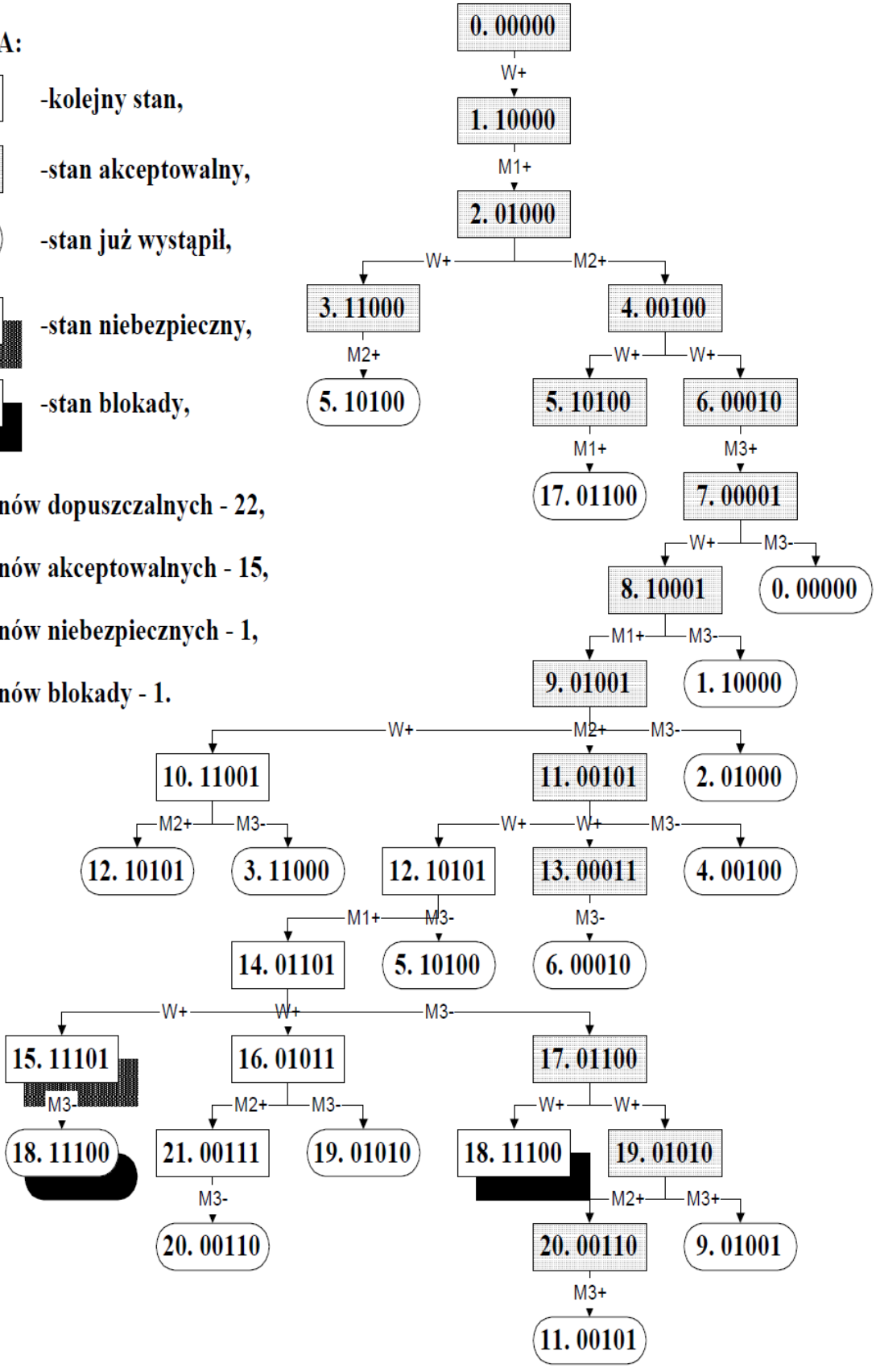
18. 11100 -stan blokady,

Liczba stanów dopuszczalnych - 22,

Liczba stanów akceptowalnych - 15,

Liczba stanów niebezpiecznych - 1,

Liczba stanów blokady - 1.



Podsumowanie

1. Przedstawione algorytmy oparte są na warunkach wystarczających dla unikania blokad i dlatego mogą odrzucać niektóre stany bezpieczne.

2. W przypadku systemu EGP najwięcej stanów akceptuje algorytm strefowy (17 stanów z ogólnej liczby 20 stanów bezpiecznych), następnie algorytm grafowy - 15 stanów, algorytm bankiera - 10 stanów, oraz algorytm „wszystko albo nic” - 6 stanów.

3. Jeżeli liczba operacji realizowanych przez procesy jest niewielka oraz nie ulega zmianie w czasie działania systemu, to można wyznaczyć całą przestrzeń stanów bezpiecznych w trybie off-line.

4. W celu zwiększenia liczby stanów akceptowalnych można stosować test „kaskadowy” złożony ze znanych algorytmów uporządkowanych według pewnego klucza (np. rosnącej złożoności obliczeniowej). W przypadku, gdy określony algorytm odrzuci badany stan wykorzystywany jest kolejny test.